3.1. Виртуальная память ОС

От адреса Go до физической RAM

адрес

виртуальная страница → PTE → физическая страница

резидентность

Сбой страницы → RAM / файл / подкачка

возврат памяти

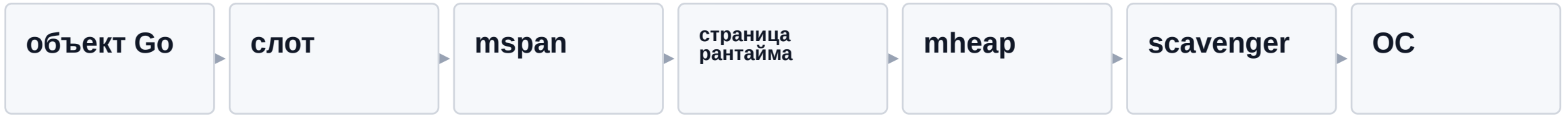
GC → sweep → scavenger → ОС

эксплуатация

RSS · cgroup · GOMEMLIMIT · OOM

Go управляет объектами и страницами рантайма. Ядро управляет физической RAM. Между ними лежит виртуальная память.

Где заканчивается Go и начинается ОС



Язык рантайма

объект · слот · span · страница рантайма · хип

Язык ОС

процесс · виртуальный адрес · отображение · страница · физическая память · права доступа · резидентность

После scavenger начинается другой уровень: виртуальная память ядра.

Мертвый объект еще не означает свободную RAM

1
объект мертв

2
слот свободен

3
RAM доступна системе

объект недостижим

GC не отметил

sweep освободил слот
слот готов к повторному использованию

span может стать пустым

страницы рантайма свободны

scavenger сообщает ОС
физические страницы можно вернуть

ядро решает судьбу страниц
резидентность / вытеснение

Освобождение объекта, повторное использование слота и возврат физической памяти - разные события.

Зачем процессу виртуальная память

инструкция CPU
↓
виртуальный адрес
↓
преобразование адреса
↓
физический адрес

- изоляция процессов
- защита страниц и режимов доступа
- ленивое выделение физической памяти
- отображаемые файлы и копирование при записи
- независимое управление адресами и резидентностью
- подкачка и вытеснение

Указатель программы обозначает виртуальный адрес. Физическая RAM под ним может меняться или временно отсутствовать.

Четыре слоя, которые нельзя смешивать

VSS

может быть огромным

виртуальное адресное пространство

какие адреса вообще существуют для процесса

↓

отображение

что стоит за диапазоном адресов

↓

обеспечение / commit

какое обеспечение система обещала

↓

резидентная физическая память

что прямо сейчас находится в RAM

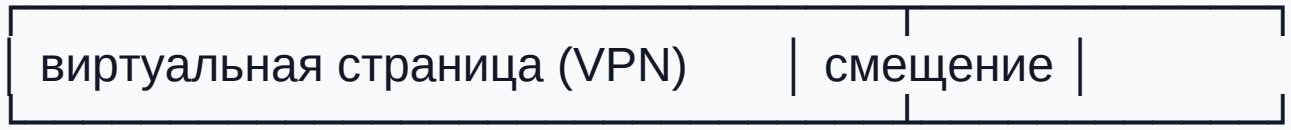
RSS

про физическую резидентность

Большой виртуальный диапазон сам по себе почти ничего не говорит о реально занятой RAM.

Виртуальный адрес: номер страницы и смещение

виртуальный адрес



Типичная учебная страница

4 KiB = 4096 B = 2^12

Следствие

для страницы 4 KiB смещение занимает 12 младших бит

ОС и MMU работают страницами: переводить каждый байт отдельно было бы бессмысленно.

Таблица страниц: адресация и права доступа

виртуальная страница
VPN

→

запись таблицы страниц
PTE

→

физический кадр
PFN

присутствует?
страница доступна

R / W / X
права доступа

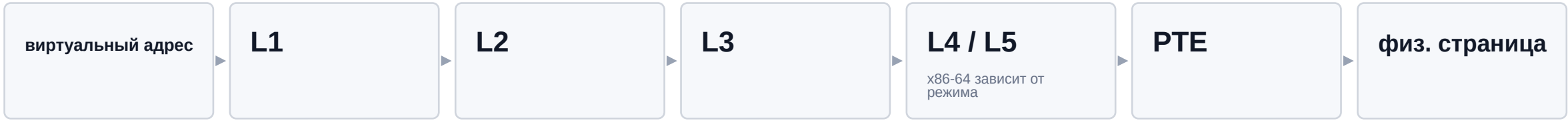
пользователь?
режим доступа

обращались
было обращение

изменена
страница менялась

Таблица страниц одновременно решает задачу адресации и задачу защиты памяти.

Почему таблицы страниц многоуровневые



Проблема плоской таблицы

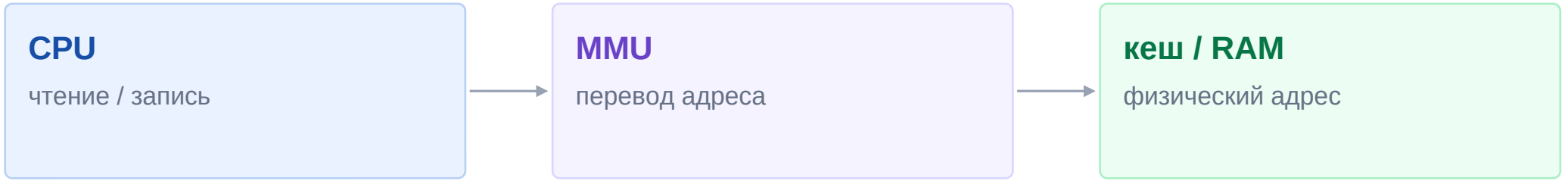
огромное виртуальное пространство потребовало бы огромной таблицы даже для пустых диапазонов

Многоуровневая структура

нижние уровни появляются только там, где реально нужны отображения

Цена: обход таблиц тоже читает память. Как избежать его при каждом обращении?

MMU делает перевод адресов аппаратно



Программа

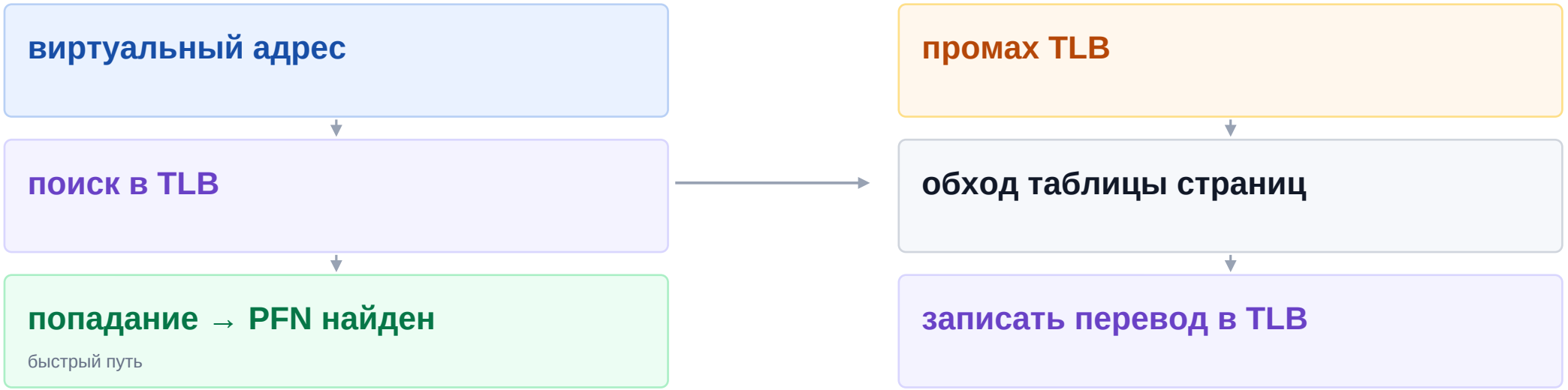
не вызывает translateAddress() вручную

Аппаратный путь

перевод участвует в выполнении каждой инструкции памяти

Аппаратная MMU устраняет программный перевод, но не цену обхода таблиц.

TLB - кеш переводов адресов



TLB хранит недавние VPN → PFN переводы и резко сокращает число обходов таблиц страниц.

Промах TLB и кеша данных — разные проблемы

Промах TLB

CPU еще выясняет, какой физический кадр соответствует виртуальному адресу.

Цена: обход таблиц и заполнение TLB.

Промах кеша данных

Адрес уже известен, но нужной кеш-линии нет в L1/L2/L3.

Цена: поход на следующий уровень памяти.

Одно обращение может потребовать и перевода адреса, и загрузки данных.

Размер страницы меняет давление на TLB

Страница 4 KiB

1 запись TLB покрывает 4 KiB.

Большой случайный рабочий набор быстро вытесняет записи TLB.

Огромная страница 2 MiB

1 запись TLB покрывает в 512 раз больше адресов.

Меньше записей перевода.

Цена

грубее шаг освобождения; внутренние потери; сложнее управление и разделение / слияние.

Огромные страницы снижают нагрузку на TLB ценой грубого управления памятью.

THP: автоматическое укрупнение страниц

много соседних обычных страниц

↓

Прозрачные огромные страницы

↓

крупное отображение

↓

меньше записей TLB

Плюс

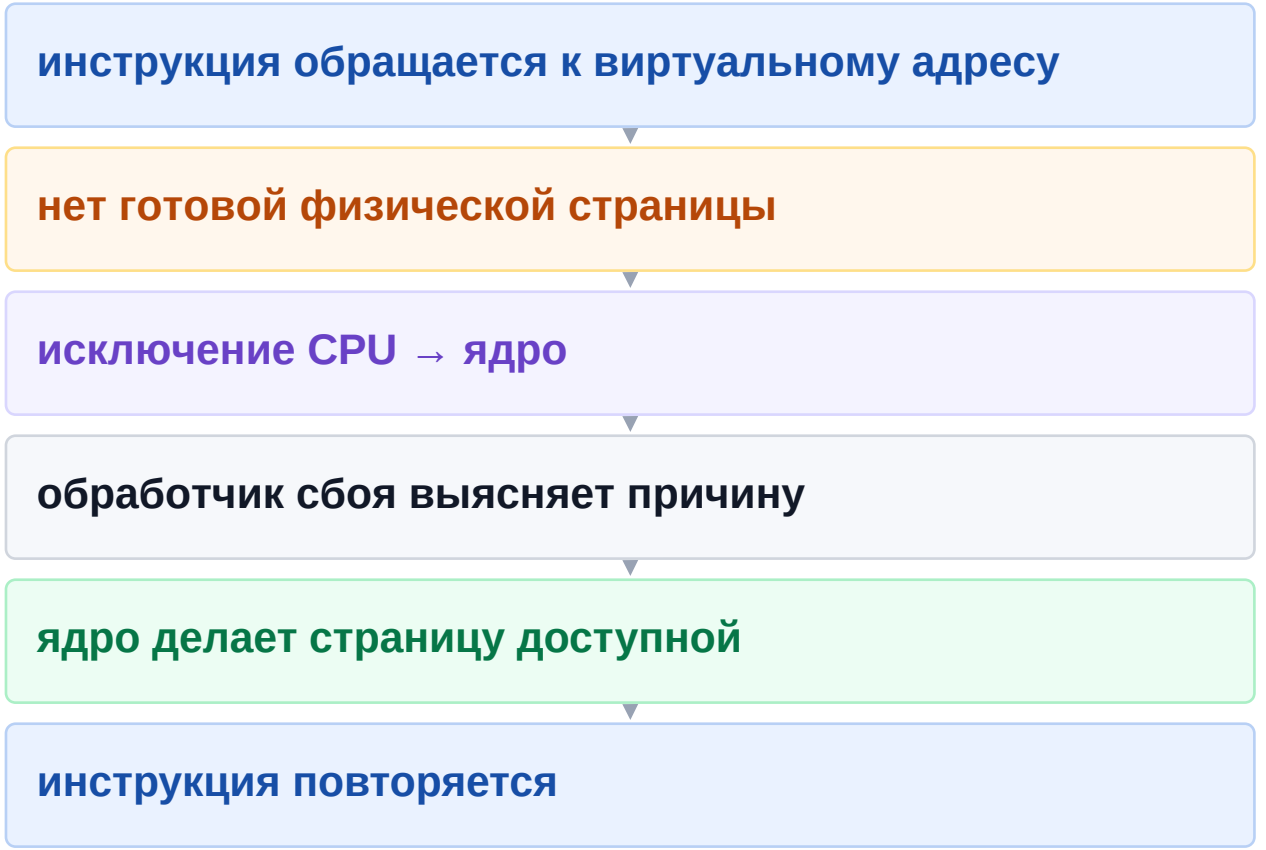
меньше промахов TLB и обходов таблицы страниц

Минус

грубый шаг освобождения и стоимость разделение / слияние

THP — компромисс ОС, а не обязательная настройка для production.

Страничный сбой — штатный механизм памяти



Страничный сбой означает, что быстрый доступ к странице сейчас невозможен. Причина может быть штатной.

Страничный сбой: продолжение или SIGSEGV

Разрешимый сбой

- первое обращение к анонимной памяти
- копирование при записи
- страница уже в страничном кеше
- нужно обновить PTE

Ядро исправляет ситуацию и повторяет инструкцию.

Недопустимое обращение

- адрес не отображен
- запись в область только для чтения
- нарушение прав доступа

Ядро не может продолжить доступ → например SIGSEGV.

Одинаковое аппаратное исключение может быть нормальной частью VM или настоящей ошибкой программы.

Малый и большой страничные сбои

Малый сбой

Дисковый I/O не требуется.

Примеры: первое касание, COW, нулевая страница; данные уже в страничном кеше, изменение PTE.

Цена: переход в ядро + метаданные + изменение PTE.

Большой сбой

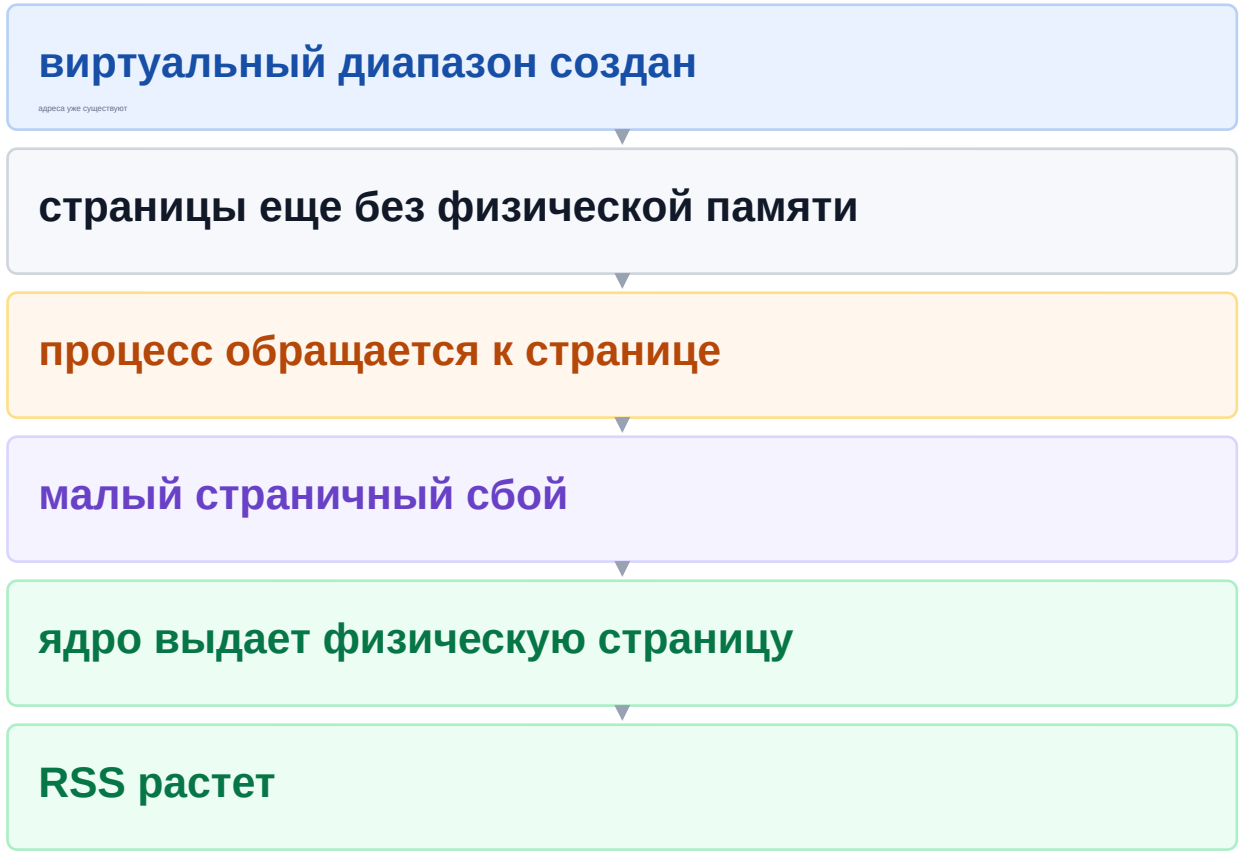
Нужен внешний I/O.

Примеры: страница файла отсутствует в RAM; анонимную страницу надо вернуть из подкачки.

Цена: задержка накопителя → рост p99.

Ключевой водораздел - требуется ли реальный I/O для восстановления страницы.

Ленивая загрузка страниц при первом обращении



Создать адресный диапазон дешевле, чем сразу выделить и обнулить всю RAM.

Почему большой make не обязан сразу дать такой же RSS

```
buf := make([]byte, 4<<30)
```

После `make`

адреса есть; физические страницы еще могут отсутствовать

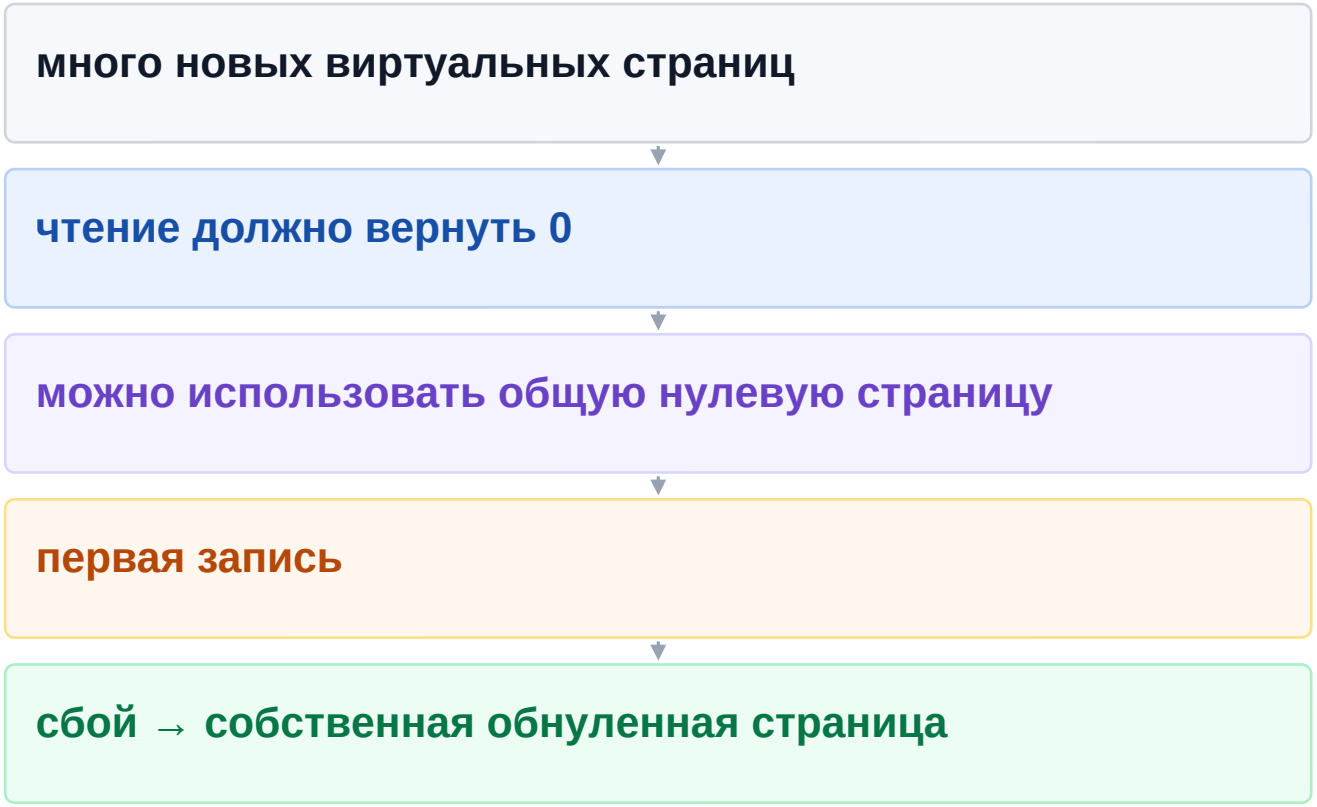
```
for i := 0; i < len(buf); i += 4096
{
    buf[i] = 1
}
```

После первого касания

сбой → физическая страница → RSS растет

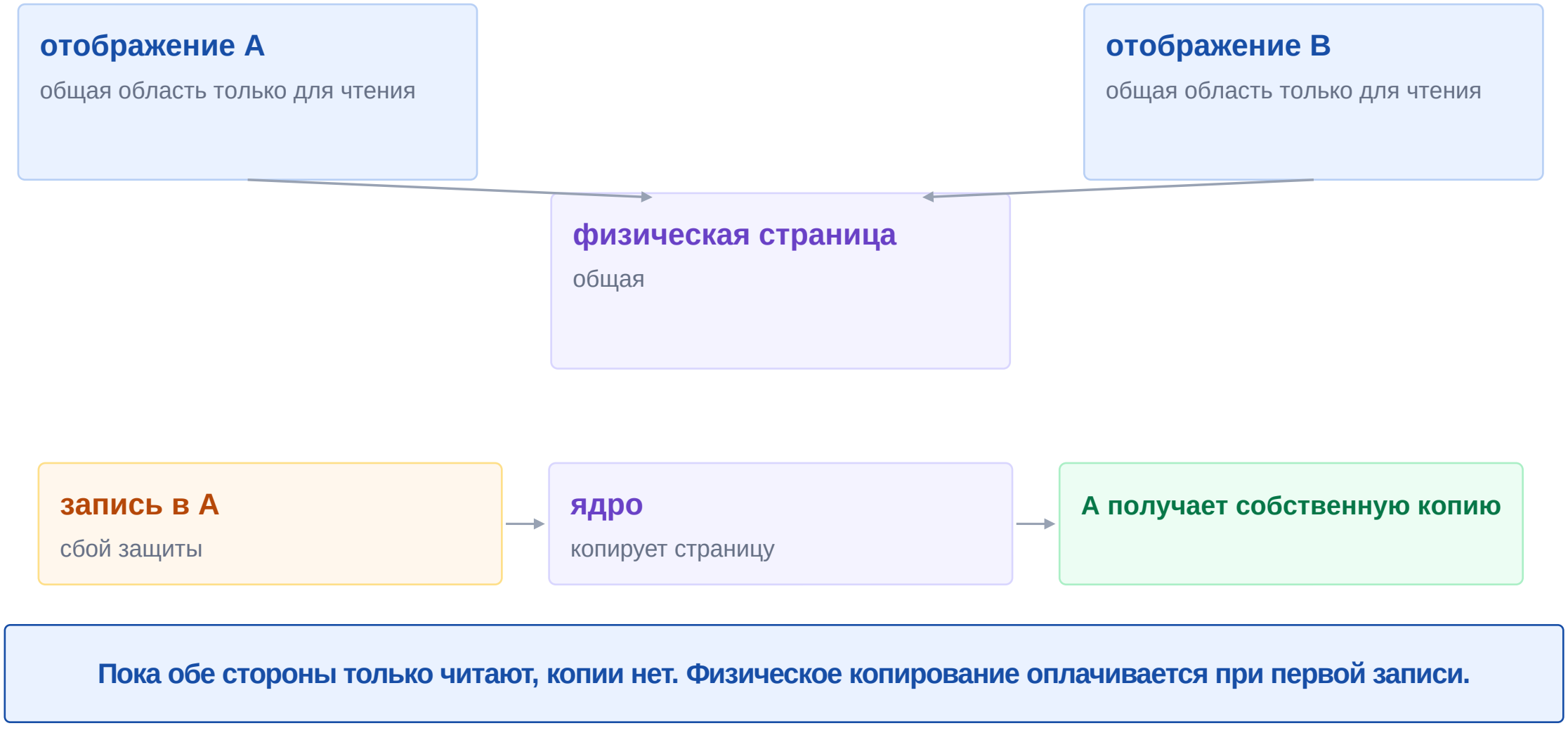
Эксперимент показывает: виртуальная память ≠ занятые страницы RAM.

Нулевая страница: нули без массовой записи



Нулевая память не требует заранее записать нули во все виртуальные страницы.

Копирование при записи откладывает расходы



Пока обе стороны только читают, копии нет. Физическое копирование оплачивается при первой записи.

Анонимная и файловая память: разное поведение

Анонимная

хип · стеки · анонимный mmap

Файла нет. Для вытеснения измененной страницы нужна подкачка или другое хранилище.

Файловая

программы · библиотеки · mmap · страничный кеш

Чистую страницу можно отбросить: данные уже есть файле и позже перечитается.

Цена освобождения зависит от внешнего источника содержимого страницы.

Страничный кеш использует свободную RAM

read()

↓

ядро

↓

поиск в страничном кеше

↓

попадание → RAM

быстро

промах страничного кеша

↓

чтение с накопителя

↓

заполнение кеша

↓

данные процессу

Занятая RAM может содержать полезный, легко освобождаемый кеш файлов. Свободный объем не дает полной картины.

Резидентная память и RSS

Резидентная страница

За виртуальной страницей сейчас стоит физическая RAM.

Если страница вытеснена, новое обращение может вызвать страничный сбой.

RSS может включать

Хип Go · стеки горутин · данные рантайма · код · библиотеки · отображенные файлы · анонимные области · память native/cgo · общие страницы с особенностями учета

RSS описывает процесс на уровне ОС. HeapAlloc описывает только один компонент памяти процесса.

Как Go доходит до VM API ОС



Аллокатор Go использует общий интерфейс; системные вызовы и семантика зависят от ОС.

Linux: `mmap` создает отображение, `munmap` удаляет

mmap

Создает диапазон виртуальных адресов.

Область может быть анонимной или файловой, общей или частной.

Само отображение не гарантирует наличие страниц в RAM.

munmap

Удаляет отображение из адресного пространства.

Старый виртуальный адрес больше не относится к прежней памяти.

Scavenger часто должен сохранить адреса, но освободить физические страницы. Нужен другой механизм.

Linux: `madvise` и судьба физических страниц



Scavenger сохраняет адреса дольше, чем связанные с ними физические страницы.

MADV_DONTNEED и MADV_FREE: две учебные модели

MADV_DONTNEED

Сильный сигнал: содержимое диапазона можно отбросить.

Следующее обращение получит заново созданную страницу.

Так работает быстрое освобождение физических страниц.

MADV_FREE

Более ленивый сигнал: данные можно выбросить, когда ядру понадобится память.

Физические страницы могут остаться до нехватки RAM.

Поэтому «Go уже освободил страницы» и «RSS прямо сейчас упал» могут расходиться во времени.

Почему RSS после scavenger падает не сразу

- тип страниц
- нехватка RAM на хосте
- активность страницы
- учет cgroup
- режим madvise

Go отметил страницы свободными

↓

scavenger сообщил ядру

↓

ядро учитывает общую нехватку RAM

↓

политика вытеснения + cgroup + другие процессы

↓

резидентность меняется по решению ядра

Метрика Go и RSS измеряют разные уровни одной цепочки, поэтому не обязаны двигаться синхронно.

Windows полезно мыслить тремя стадиями

Резервирование адресов

адреса закреплены

→

Commit

система гарантирует обеспечение

→

Working Set

страницы сейчас в RAM

Главное разделение

резерв адресов ≠ Commit ≠ страницы в RAM

В Windows особенно четко разделены резерв адресов, обеспечение и присутствие в RAM.

RESERVE → COMMIT → DECOMMIT → RELEASE

MEM_RESERVE
виртуальный диапазон

→

MEM_COMMIT
доступная память

→

MEM_DECOMMIT
снять память, оставить адреса

→

MEM_RELEASE
освободить и адреса

DECOMMIT
модель «адреса остаются, физическое обеспечение больше не нужно»

RELEASE
диапазон больше не принадлежит процессу

Windows API явно разделяет виртуальные адреса и обеспечение памятью.

Working Set, Commit, Private Bytes: разные метрики

Working Set

Какие страницы процесса прямо сейчас находятся в физической RAM?

Commit

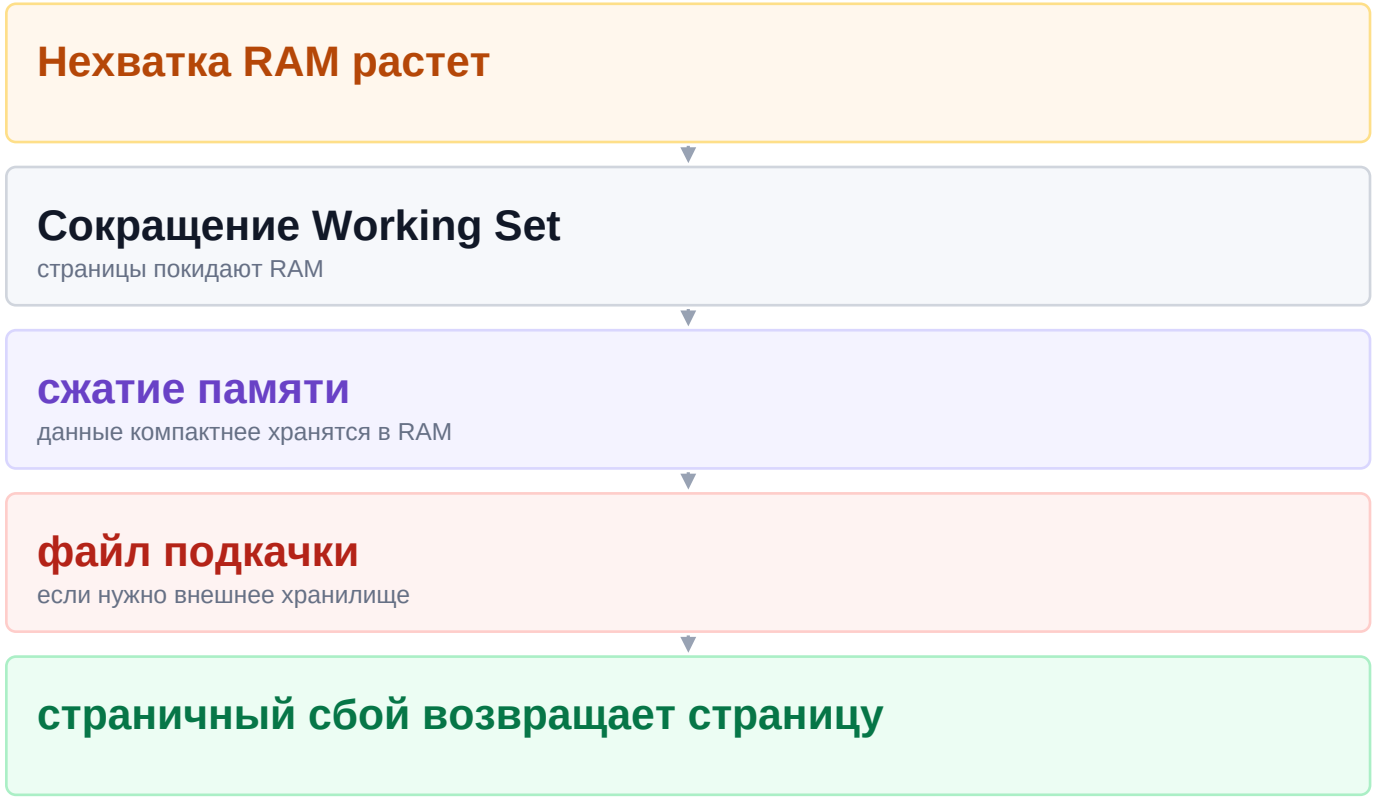
Сколько памяти система обязалась предоставить?

Private Bytes

Сколько частной выделенной памяти относится к процессу?

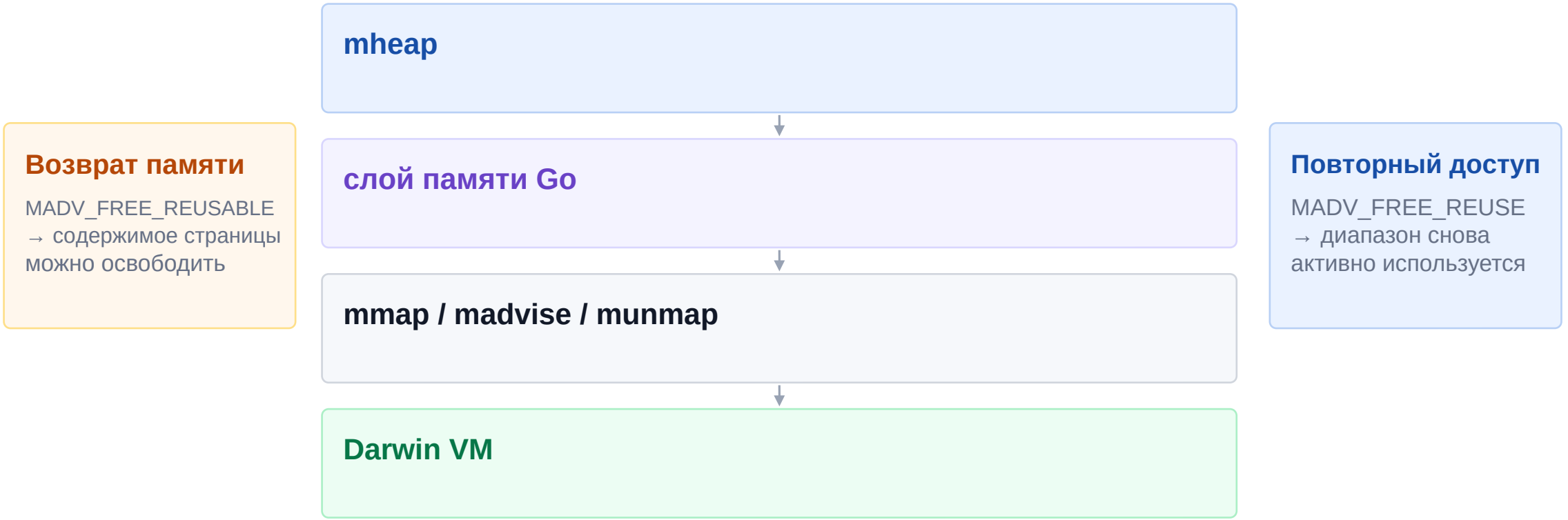
Большие Commit и Working Set означают разные состояния и требуют разной диагностики.

Windows: давление памяти → сжатие → файл подкачки



Сокращение Working Set не доказывает освобождение памяти приложением: это могла сделать ОС.

macOS: та же модель памяти через Darwin



На macOS виртуальные адреса живут отдельно от их физических страниц.

macOS: давление памяти важнее свободного объема

Нехватка RAM
растет

→

сжатие
меньше данных в RAM

→

подкачка
если сжатия недостаточно

→

страничный сбой
возврат данных

Memory Pressure
Учитывает доступность RAM, сжатие, подкачку и возможность обслуживать новые запросы.

Цена сжатия
Экономим RAM, тратим CPU на сжатие и распаковку.

Мало свободной RAM на macOS — норма; опасны высокое давление памяти и подкачка.

Одна задача виртуальной памяти - три набора API

Задача	Linux	Windows	macOS
Создать диапазон адресов	mmap	MEM_RESERVE	mmap
Сделать память доступной	отображение + сбои	MEM_COMMIT	отображение + сбои
Физическое обеспечение не нужно	madvise	MEM_DECOMMIT	MADV_FREE_REUSABLE
Удалить отображение	munmap	MEM_RELEASE	munmap
Метрика резидентности	RSS	Working Set	резидентная память

Общая причинная модель одна; системные API и политика конкретной ОС различаются.

Какую страницу ядру выгоднее освободить?

RAM становится тесно

↓

ядро выбирает страницы

↓

файловая, чистая?

Да

можно отбросить: файл уже содержит данные

Измененная файловая

нужна запись перед освобождением

Анонимная

нужна подкачка или другое хранилище

Цена освобождения зависит от происхождения страницы и наличия изменений.

Чистая файловая страница — дешевый кандидат

чистая страница из файла

↓

данные уже существуют в файле

↓

физическую страницу можно отбросить

↓

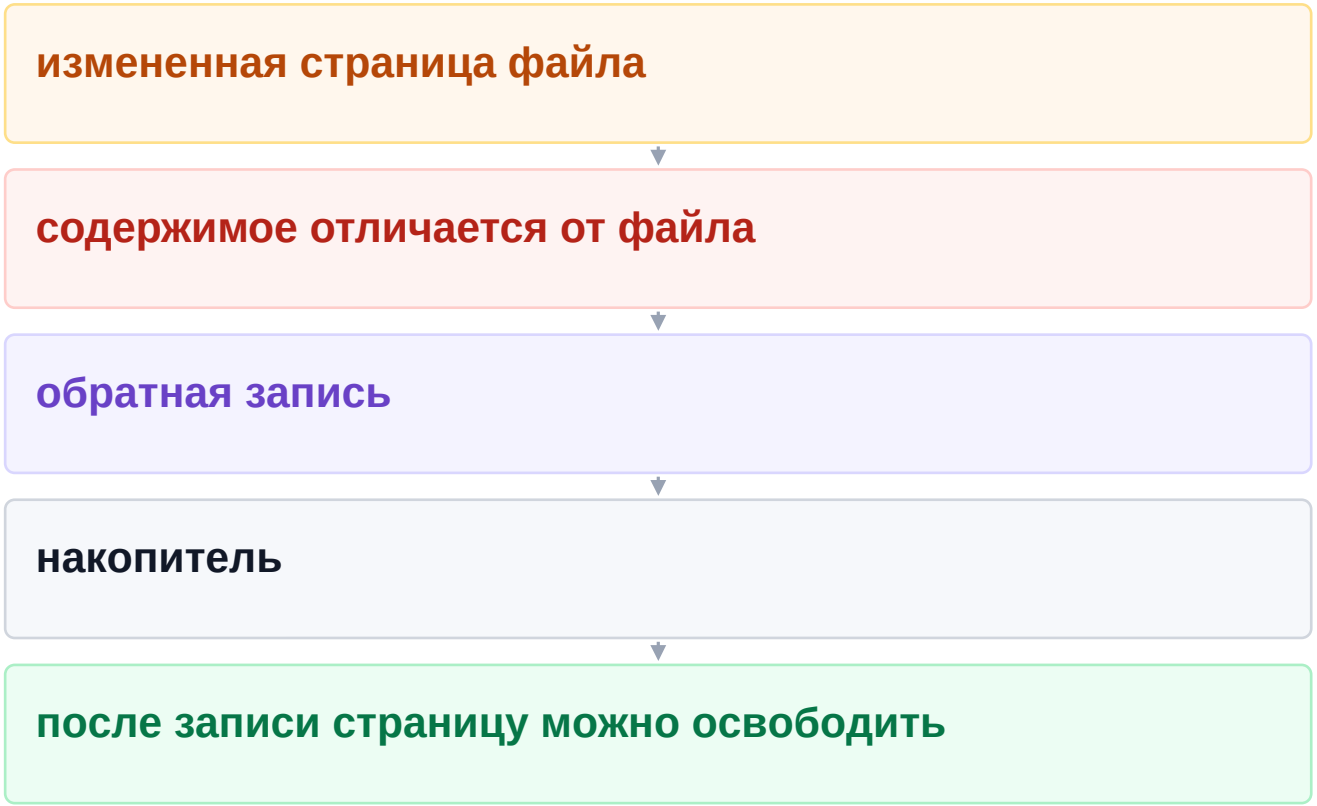
следующее обращение

↓

чтение файла / заполнение кеша

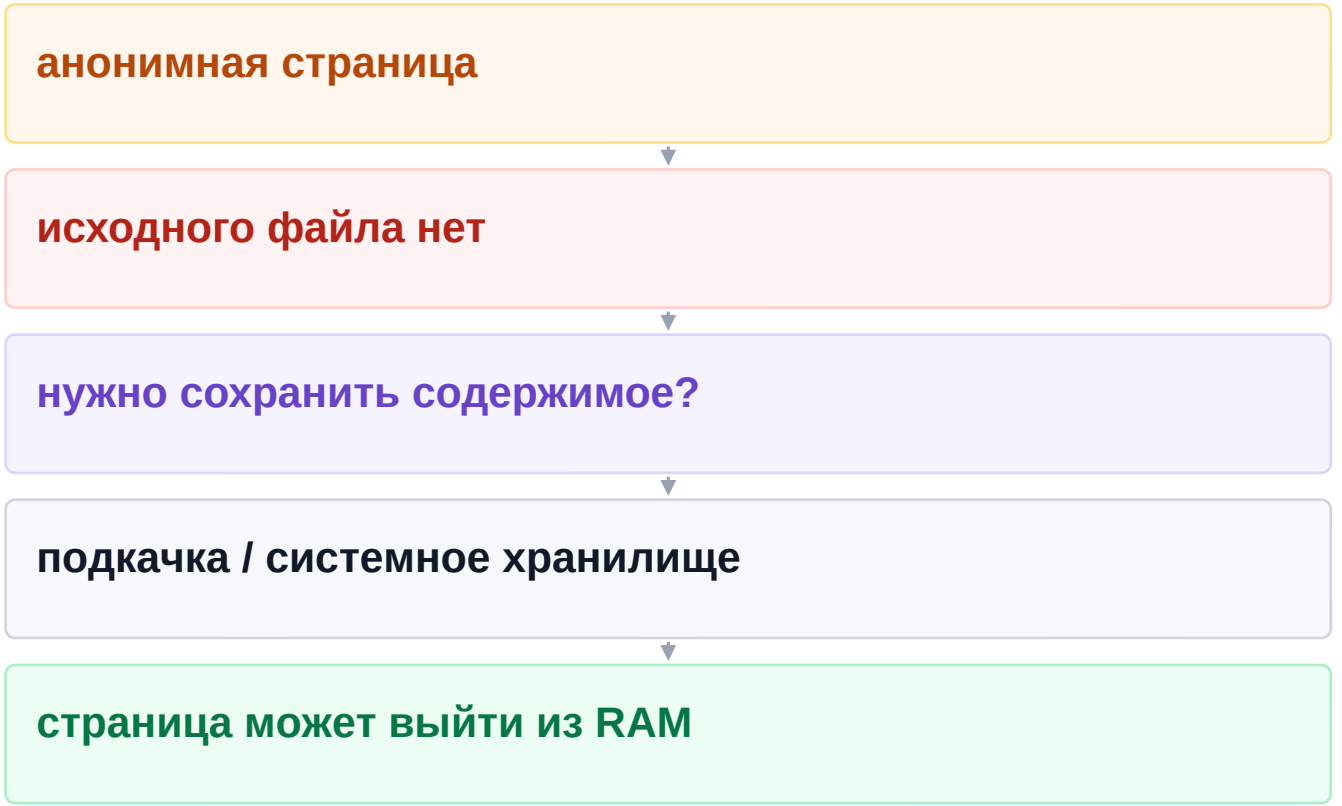
Чистый страничный кеш можно освободить без предварительной записи на диск.

Измененная файловая страница требует записи



Виртуальная память связана с накопителем: измененные страницы → запись → хранение.

Анонимную страницу нельзя перечитать из файла



Анонимную память освободить сложнее: системе нужно сохранить уникальное содержимое.

Подкачка — часть политики управления памятью

Слишком простая модель

«Подкачка появляется лишь когда RAM полностью закончилась».

Она скрывает упреждающее вытеснение и баланс между кешем, анонимной памятью и будущими запросами.

Полезная модель

Горячая анонимная страница вытеснена → снова нужна → большой сбой → чтение с диска → большая задержка.

Важен не сам факт подкачки, а попадание рабочего набора в медленный путь ввода-вывода.

Пробуксовка: рабочий набор больше доступной RAM

рабочий набор > доступная RAM

↓

страницы вытесняются

↓

через короткое время снова нужны

↓

большие страничные сбои

↓

возврат страниц → вытеснение других

↓

цикл повторяется

Система начинает тратить ресурсы на движение страниц вместо полезной работы.

Как пробуксовка памяти выглядит снаружи

Задержка	Поток запросов	Ввод-вывод	Большие сбои	Подкачка
p95/p99 резко растут	падает	высокий	растут	активен

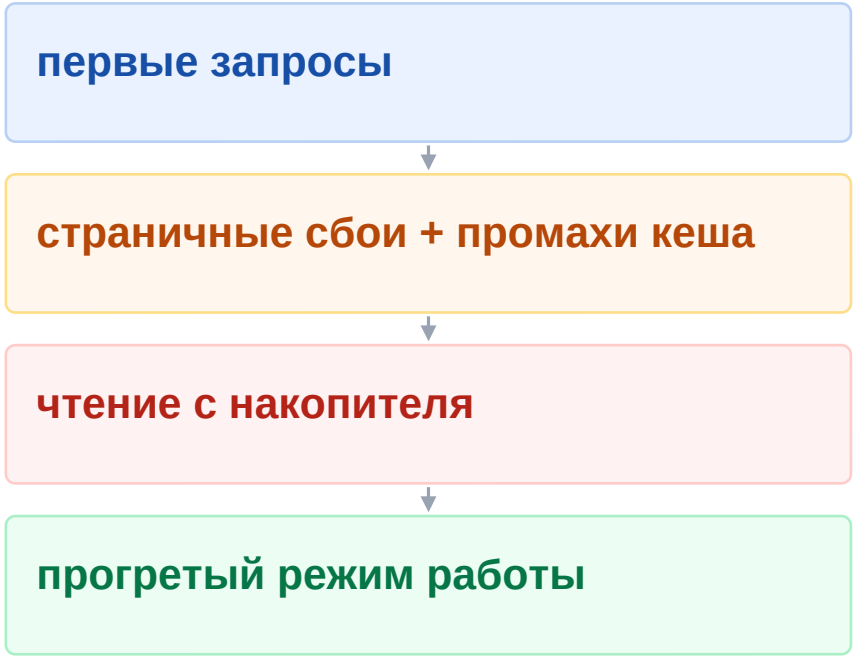
CPU может выглядеть «странно»

CPU может ждать восстановления страниц и дисковых операций.

При нехватке памяти ищем цепочку: вытеснение → сбои/диск → задержки.

Холодный старт: память пока не прогрета

- страничный кеш холодный
- TLB холодный
- кеши CPU холодные
- часть анонимных страниц еще не трогали
- страницы файлов еще не прочитаны



После рестарта задержка может отличаться от устойчивого режима при том же коде.

NUMA добавляет вопрос о близости физической страницы



Виртуальная память выбирает физическую страницу; NUMA определяет стоимость доступа к ней с конкретного CPU.

Контейнер с 1 GiB — не отдельный компьютер

Хост

может иметь 256 GiB RAM

процесс Go

↓

обычная виртуальная память Linux

↓

учет памяти cgroup

↓

лимит памяти

cgroup

процессу разрешен свой бюджет

Cgroup добавляет учет и ограничения поверх общей виртуальной памяти Linux.

Как cgroup учитывает память

процесс использует страницы

▼

ядро учитывает память в cgroup

▼

расход приближается к лимиту

▼

вытеснение / нехватка памяти

▼

жесткая граница

▼

возможно событие OOM

Для контейнера решающим является учет ядра, а не только метрики Go.

HeapAlloc ниже memory.max еще не означает запас

HeapAlloc

600 MiB

memory.max

1 GiB

«запас»?

400 MiB только в слишком простой модели

- стеки горутин
- данные рантайма
- выделения native/cgo
- mmap
- учет файловой памяти и кеша
- фрагментация и всплески

Контейнерный лимит считает больше, чем живые Go-объекты.

GOMEMLIMIT и memory.max: разные уровни контроля

GOMEMLIMIT

Мягкий лимит памяти рантайма Go.

Рантайм меняет частоту GC и пытается удерживать управляемую им память около бюджета.

memory.max

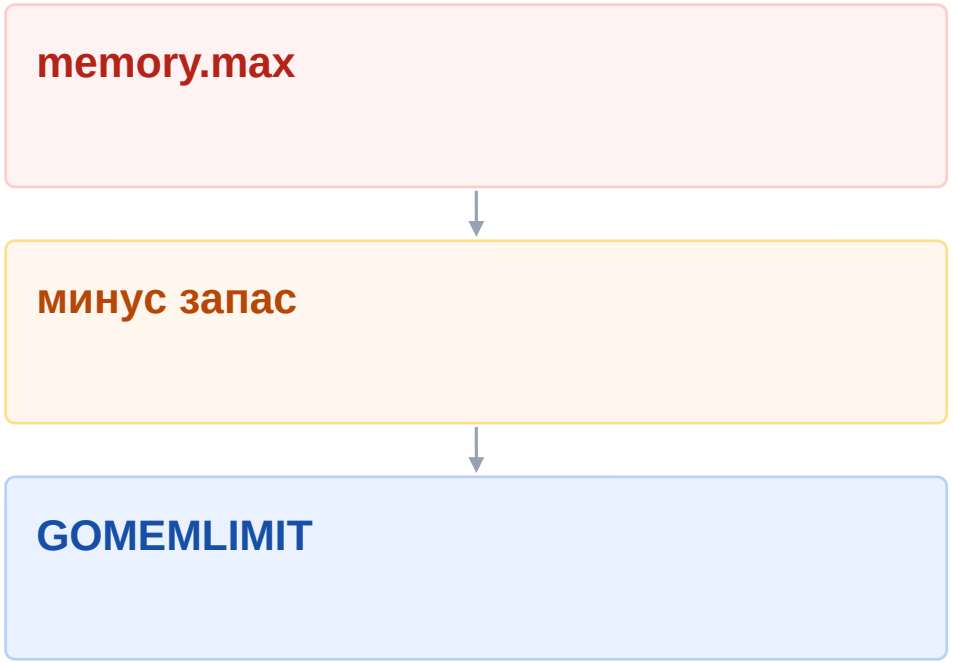
Жесткая граница cgroup.

При превышении ядро запускает вытеснение / OOM механизмы.

Если GOMEMLIMIT равен memory.max, память вне Go и всплески быстро израсходуют запас.

Зачем запас между мягким и жестким лимитом

- память cgo и native
- стеки горутин
- mmap
- файловая память и ввод-вывод
- фрагментация
- кратковременные всплески



- размер запаса определяется нагрузочными измерениями
- универсальной безопасной константы нет

Запас — бюджет на память, которую GC напрямую не контролирует.

OOM бывает двух разных классов

Ошибка внутри процесса

Рантайм запрашивает память → ОС не может выделить ее → рантайм не может продолжить выполнение → аварийное завершение.

OOM со стороны cgroup / ядра

Учет памяти → жесткий лимит → вытеснение не помогло → OOM → ядро выбирает процесс для завершения.

Похожий внешний симптом может иметь разный механизм. Диагноз начинается с того, кто именно прекратил процесс.

Почему Kubernetes показывает OOMKilled

pod / контейнер

▼

лимит памяти cgroup

▼

использование памяти процессом

▼

нехватка у жесткого лимита

▼

обработка OOM ядром

▼

процесс контейнера завершен

▼

Kubernetes получает причину завершения

Возможные причины

живые данные · вне Go · всплеск · утечка · страничный кеш и его учет · слишком малый лимит

OOMKilled означает завершение ядром при нехватке памяти, а не дефект GC.

Linux: от профиля хипа Go к памяти процесса

/proc/<pid>/status

сводка: виртуальная / резидентная память и счетчики

smaps / smaps_rollup

разбор областей: анонимные, файловые, общие и частные

pmap

какие области памяти отображены

pprof объясняет объекты Go; /proc и pmap — адресное пространство и RAM процесса.

Linux: `vmstat` и сбои показывают нехватку RAM

vmstat

Смотрим свободную RAM, подкачку, готовые задачи, I/O, системную активность.

Один снимок редко дает диагноз.

Малые и большие сбои

Малые сбои могут быть обычной активностью ОС.

Рост больших сбоев означает чтение данных с диска и возможное увеличение задержки.

Диагностика памяти - это корреляция счетчиков во времени, а не поиск одного «плохого» числа.

Cgroup v2: три файла для диагностики памяти

memory.current

Сколько памяти сейчас учитывается за cgroup?

memory.max

Какова жесткая верхняя граница?

memory.events

Были ли события нехватки, лимитов и OOM?

Для расследования OOM контейнера эти метрики ближе к реальному решению ядра, чем один HeapAlloc.

Windows: какие метрики сопоставлять

- Task Manager / Resource Monitor
- Performance Monitor
- Process Explorer
- Working Set
- Commit Size
- Private Bytes
- страничные сбои

Главный вопрос

У процесса выросло обязательство Commit или прямо сейчас вырос резидентный Working Set?

Это разные расследования.

В Windows различаем виртуальный объем, Commit и память, присутствующую в RAM.

macOS: давление памяти, сжатие и подкачка

```
vm_stat
vmmap <pid>
memory_pressure
```

- Давление памяти
- Память приложений
- Закрепленная память
- Сжатая память
- Использование подкачки

Вопрос

Системе не хватает памяти или она просто использует RAM эффективно?

На macOS объем свободной RAM сам по себе малоинформативен.

Почему мало свободной RAM — еще не проблема

Плохая цепочка

Свободно 200 MiB → «катастрофа».

Не учитывается освобождаемый страничный кеш и политика использования свободной RAM.

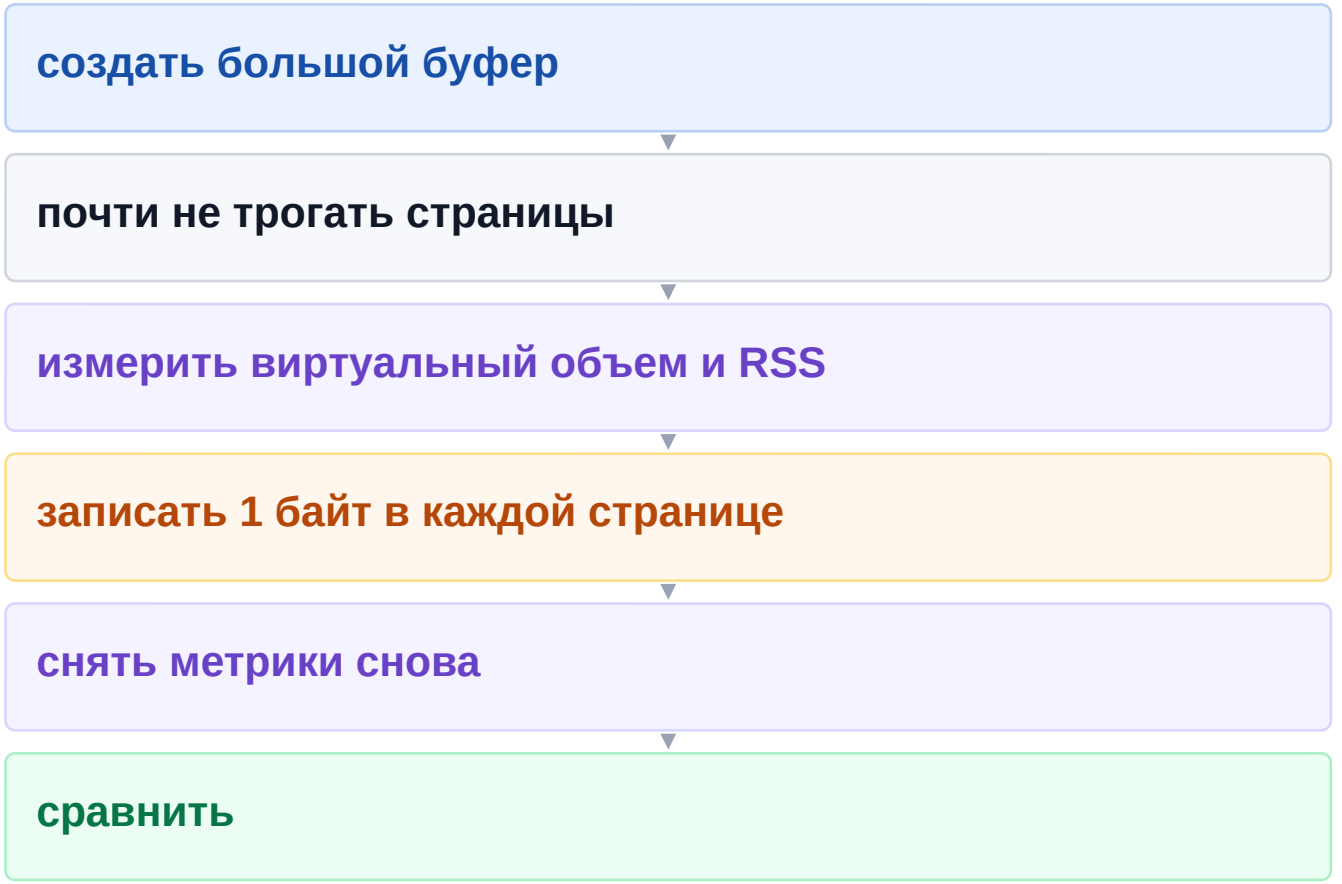
Полезная цепочка

Нехватка RAM → эффективность вытеснения → сбои → подкачка / сжатие → задержка.

Вот эти связи описывают реальную проблему.

Современная ОС старается использовать RAM. Пустая RAM сама по себе не является целью.

Практика 1: виртуальный объем ≠ RAM



Виртуальный диапазон появляется раньше, чем страницы начинают занимать RAM.

Практика 2: первое касание и страничные сбои

Первый проход

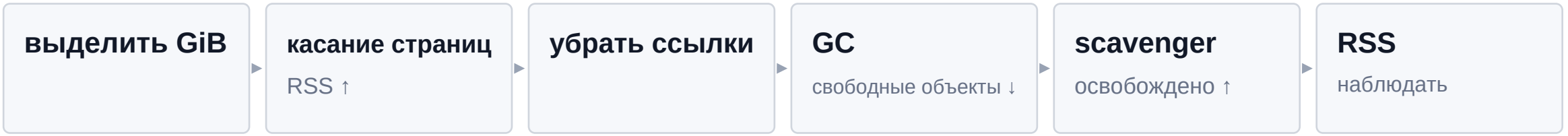
Страниц нет в RAM → сбои → выделение памяти → больше времени.

Второй проход

Страницы уже в RAM → меньше работы → быстрее.

Цель — измерить цену создания физических страниц, а не просто сравнить два числа.

Практика 3: GC, scavenger и RSS работают поэтапно



- Go heap objects
- heap free
- heap released
- RSS / Working Set

Что доказать

объект мертв → слот свободен → страница возвращена → RSS падает: четыре разные стадии

Практика должна сломать автоматическую связь «GC прошел → RAM сразу вернулась ОС».

Практика 4: GOMEMLIMIT и жесткий лимит cgroup

одинаковая нагрузка → **одинаковый memory.max** → **разные GOMEMLIMIT**

- живой хип / цель GC
- циклы GC / помощь приложений
- RSS
- memory.current / memory.events
- p95 / p99

Что доказать

Мягкий лимит Go влияет на GC, а жесткий лимит остается отдельным решением ядра.

Один лимит cgroup может дать разную задержку/CPU при разных GOMEMLIMIT.

Практика 5: нехватка памяти увеличивает задержку



Показать два режима одного кода: памяти достаточно и памяти не хватает.

Практика 6: страничный кеш ускоряет чтение

Первое чтение

read → промах кеша → накопитель → заполнение → процесс

Повторное чтение

read → попадание в кеш → RAM → процесс

Страничный кеш связывает память, файловый ввод-вывод и последующую тему writeback/fsync.

Пять пар метрик, которые нельзя смешивать

Слева	Справа	Почему разные
Виртуальный объем	RSS	диапазон адресов / страницы в RAM
RSS	живой хип Go	вся память процесса / объекты Go
Commit	Working Set	обеспечение памяти / присутствие в RAM
Свободная RAM	Давление памяти	снимок свободной RAM / способность освобождать
Освобождение GC	Вытеснение ядром	состояние объектов Go / физическая RAM

Сначала сформулируйте вопрос, потом выбирайте метрику. Иначе числа будут спорить друг с другом.

Сквозная цепочка: выделение памяти

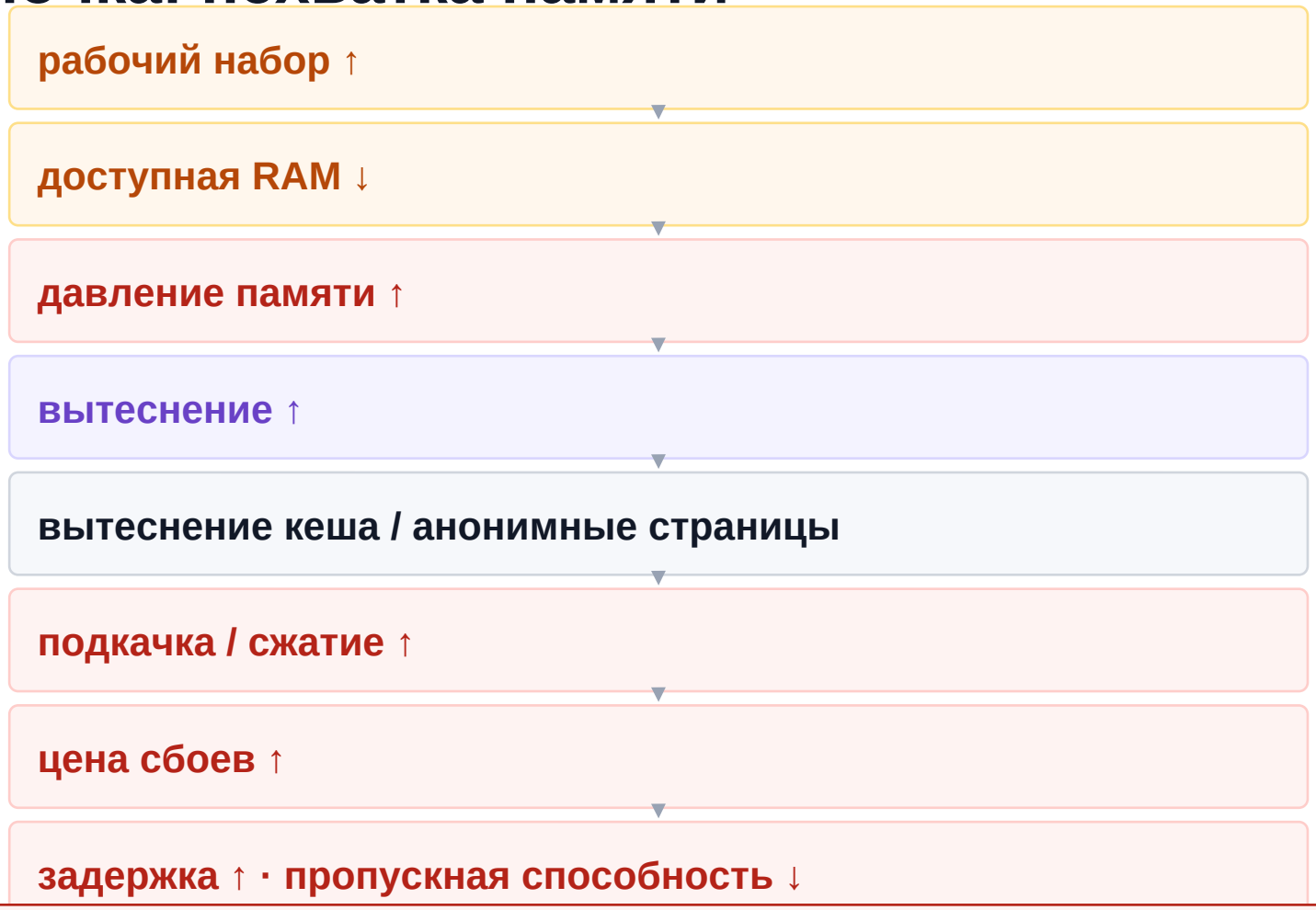


Сквозная цепочка: освобождение памяти



Каждый слой освобождает свой ресурс; следующий слой узнает об этом позже.

Сквозная цепочка: нехватка памяти



Память становится проблемой, когда решения ОС попадают в путь задержки запроса.

Сквозная цепочка: OOM в контейнере



OOM контейнера вызван общим учетом памяти, а не только размером хипа Go.

Границы модели: что общее, что зависит от ОС

Устойчивая общая модель

виртуальный адрес → перевод → физическая страница → RAM / вне RAM → страничный сбой → вытеснение

Особенности платформ

Linux: mmap / madvise / cgroup
Windows: reserve / commit / Working Set / pagefile
macOS: mmap / повторное использование / сжатие / подкачка

Сначала общая модель, затем особенности API и политики конкретной ОС.

Что важно запомнить

- Программа использует виртуальные адреса; ядро управляет физическими страницами.
- TLB кеширует переводы; промах TLB и промах кеша данных — разные события.
- Виртуальный объем, обеспечение и присутствие в RAM — разные уровни.
- Страничный сбой штатен; большой сбой может добавить чтение с диска.
- Анонимные и файловые страницы имеют разную цену освобождения.
- RSS шире живого хипа Go; учет контейнера охватывает и другую память.
- GOMEMLIMIT — мягкий лимит Go; memory.max — жесткий лимит ядра.
- GC, scavenger, вытеснение ядром и падение RSS — разные стадии.

Диагностика памяти требует пройти от объекта Go до физической страницы и обратно.

Финальная модель: от объекта Go до RAM и обратно

Объект Go ▸ mspan ▸ страница Go ▸ виртуальный адрес ▸ таблица / TLB ▸ страница RAM ▸ RAM / файл / своп

недостижим ▸ GC ▸ sweep ▸ scavenger ▸ API памяти ОС ▸ вытеснение ▸ RAM доступна

Главный инженерный вывод

Go управляет объектами, span и страницами рантайма. Ядро управляет физической RAM. Их связывают отображения, таблицы страниц, TLB, сбои, резидентность и вытеснение.